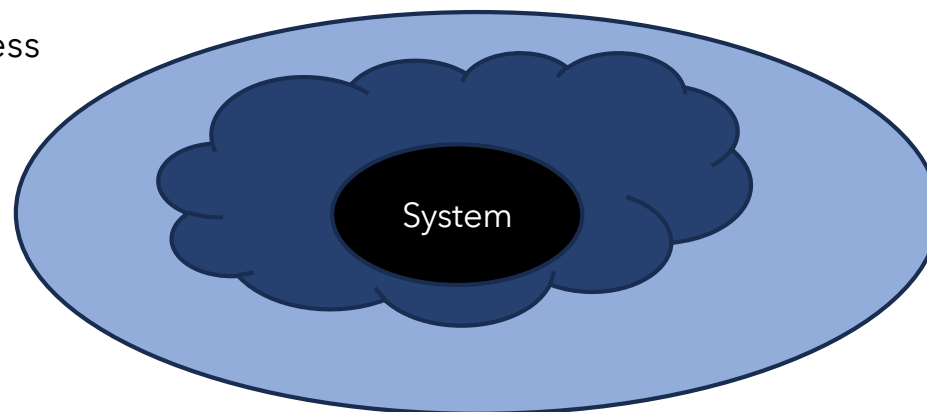




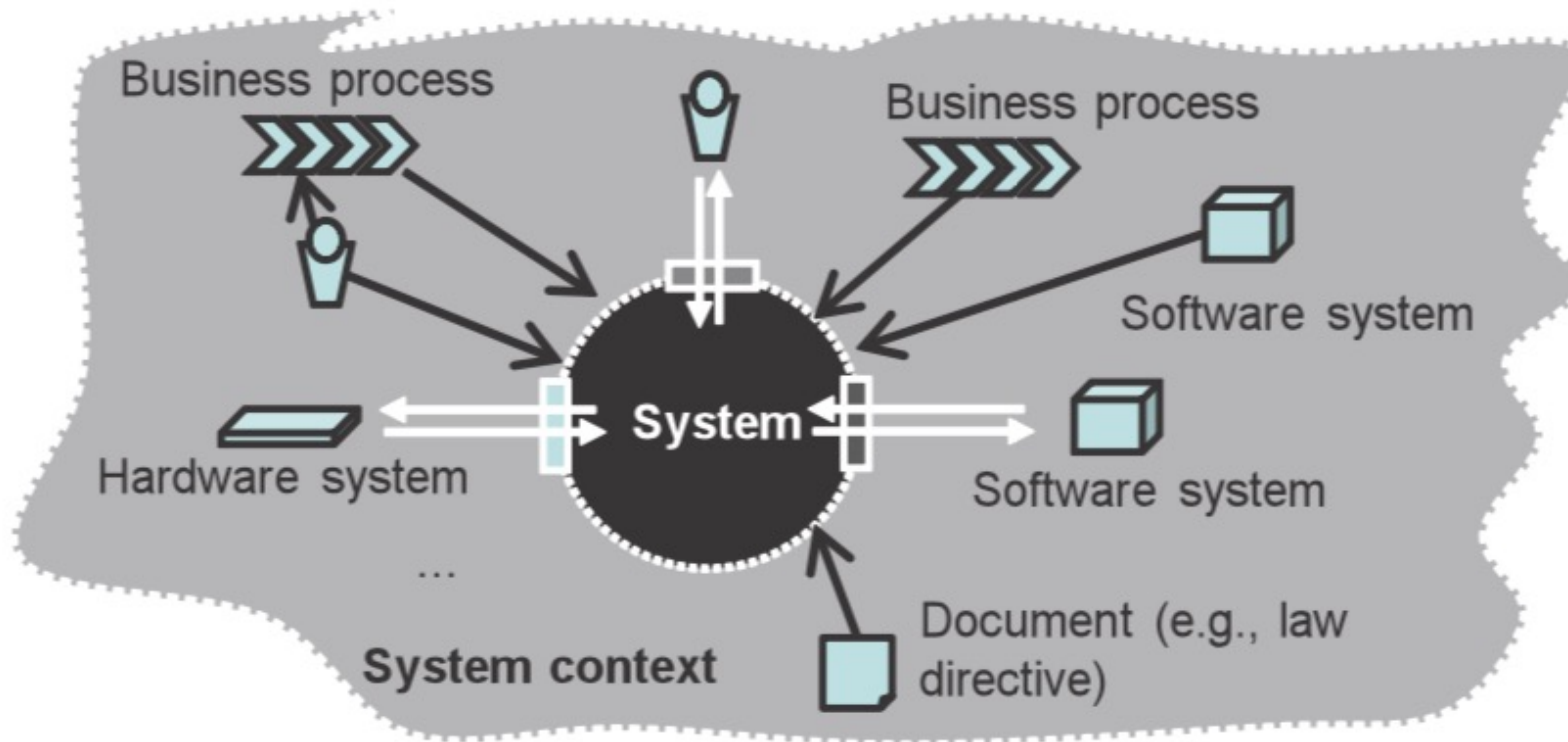
Day 2

Defining System and Context Boundaries

- System- and context boundaries awareness
- Techniques
 - Interviews
 - Observation
 - Documentation centric (existing docs / predecessor of the system)
- **FOCUS: WHAT** do we have to create (not HOW)!



System / Context Boundaries

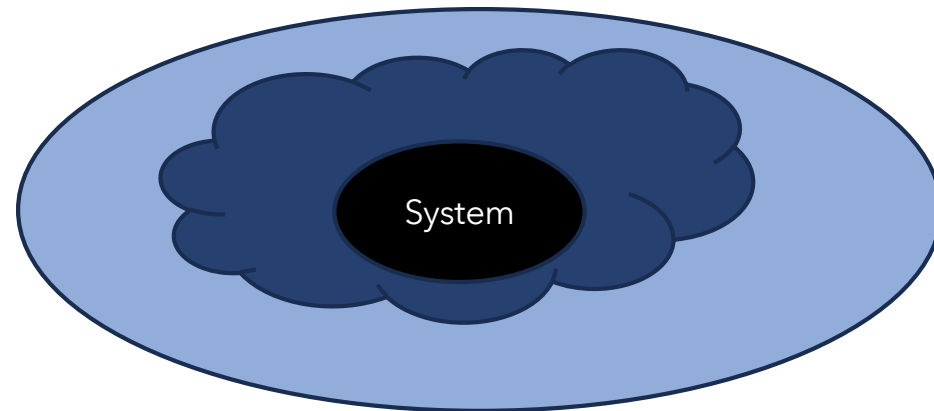


dary

System Context



- The system **context** is the **part of the system environment** that is **relevant** for the definition as well as the understanding of the requirements of a system to be developed.



[Requirements Engineering Fundamentals], p12 ff

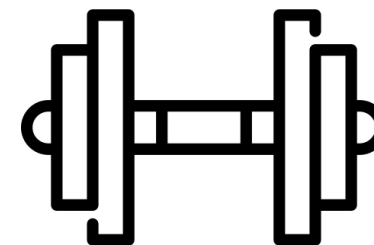
Boundary



- **Defining the system boundary:** When defining the system boundary, a decision must be made: Which aspects pertain to the system to be developed and which aspects belong in the system context?
- **Defining the context boundary:** When defining the context boundary, the question to be answered is: Which aspects pertain to the system context (i.e. have a relation to the system to be developed) and which aspects are part of the irrelevant environment?

[Requirements Engineering Fundamentals], p12 ff

Ex: Draw a System / Context boundaries for a coffee vending machine



IT-Sec Ressources

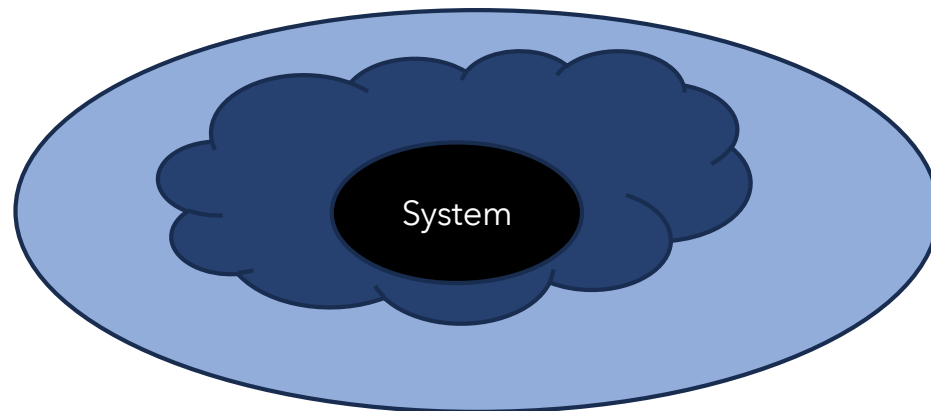


- Information Security Society Switzerland (ISSS)
 - <https://www.iss.ch/de/>
- SANS
 - <https://www.sans.org>
- (ISC)²
 - <https://www.isc2.org>

System boundaries can shift!



- Frequently, the system boundary is not precisely defined until the end of the requirements engineering process. Before that, some or several interfaces as well as desired functions and qualities of the system to be developed are only partially known or not known at all. We refer to this initially vague separation of the system and its context as the gray zone between the system and the context



[Requirements Engineering Fundamentals], p12 ff

How to „find“ these requirements?

Process & Methods

2 main activities (→ remember 4 core activities)



1. **Elicitation:** During requirements elicitation, different techniques are used to obtain requirements from stakeholders and other sources and to refine the requirements in greater detail.
 - **Documentation:** During documentation, the elicited requirements are described adequately. Different techniques are used to document the requirements by using natural language or conceptual models.
 - **Validation and negotiation:** In order to guarantee that the predefined quality criteria are met, documented requirements must be validated and negotiated early on.
2. **Management:** Requirements management is orthogonal to all other activities and comprises any measures that are necessary to structure requirements, to prepare them so that they can be used by different roles, to maintain consistency after changes, and to ensure their implementation.

Based on [ISO/IEC/IEEE 29148:2011]

Specification process



There's no fits once and for all specification process in RE

→ Tailor it to/for your special needs

Factors:

- Linear vs. agile project
- How flexible are you / how accurate does it have to be?
- Are customers and users known and can they be involved?
- Do we develop something for an internal need or a public solution?
- Should we use standard tools and techniques?

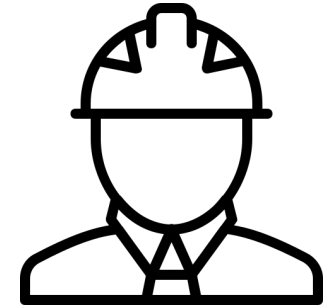
Requirement elicitation

- Start with what you get..
- List and analyze stakeholders (and their availability)
 - Who is the customer?
 - Who are the users?
 - Who are the deciders?
 - External systems or organizations involved?
 - Regulatory constraints/requirements?
- Stakeholder-Map



The requirements engineer

- speaks the **language** of the stakeholders,
- becomes thoroughly **familiar** with the application domain,
- **creates** a requirements **document**,
- can get **work results across** (e.g., by means of diagrams and graphs),
- maintains a respectful **relationship** with any stakeholder,
- **presents** their **ideas** and alternatives as well as their realizations,
- **allows** stakeholders to **demand properties** that make the system user-friendly and simple,
- **ensures** that the system satisfies the **functional and qualitative demands** of the stakeholders



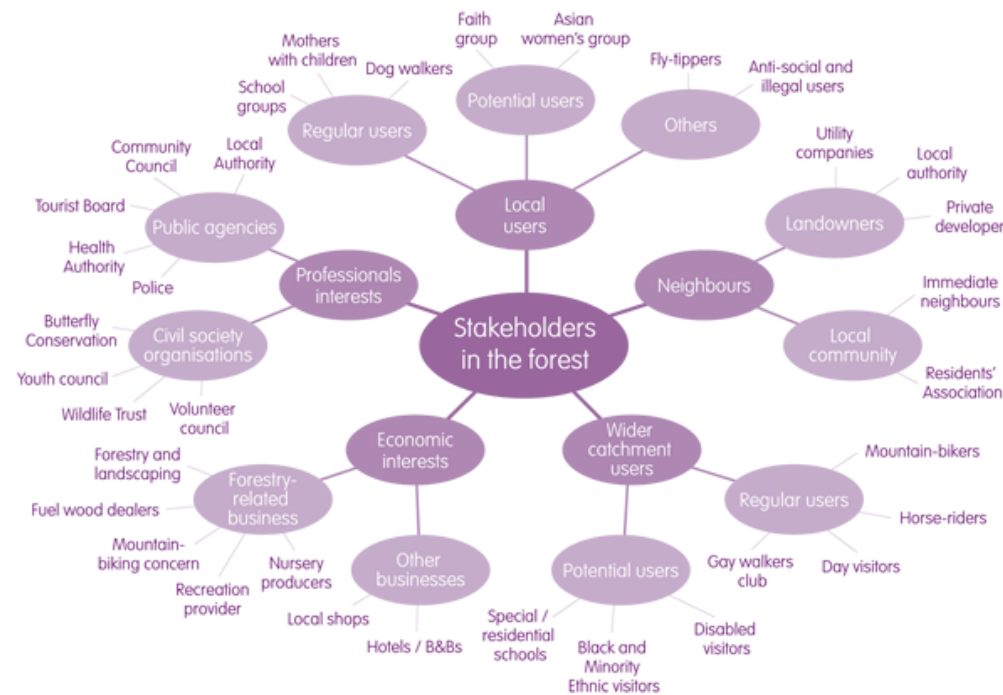
[Requirements Engineering Fundamentals], p21 ff

The stakeholders

- introduce the requirements engineer to the application domain,
- supply the requirements engineer with requirements,
- document requirements assiduously,
- make timely decisions,
- respect the requirements engineer's estimates of costs and feasibility,
- prioritize requirements,
- inspect the requirements that the requirements engineer documents, such as prototypes, etc.,
- communicate changes in requirements immediately,
- adhere to the predetermined change process,
- respect the requirements engineering process that has been instated.

[Requirements Engineering Fundamentals], p21 ff

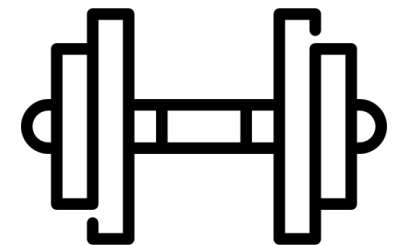
Requirement elicitation



The goal is that everyone knows who is in charge!



Ex: List stakeholders of a coffee vending machine



A sample process



Beteiligte auf
Kundenseite

Systembedürfnis

Auftrag

Anforderungs-ingenieure

Fragen beantworten,
Anforderungen nennen

Fragen

Problem studieren,
Informationsgewinnung
planen

Sehr wichtig!

Antworten, Aussagen

Ergänzende Fragen

Analysieren, auswerten

Prüfen, durchspielen

Korrekturen, Ergänzungen

Inkrementeller Aufbau der
Spezifikation

Glossare, Szenarien,
Modellfragmente

Prüfen, validieren

Fertige Spezifikation

Korrektur, Freigabe

Validierte Spezifikation

Manage Requiriements



- Ensure
 - elicitation
 - documentation
 - refinement
 - changes

of requirements is adequately dealt with during a lifecycle, with a view toward satisfying the overall mission or need in a quality manner and to the customers' satisfaction.

Roundup



Requirements Engineering means, Requirements:

- Collection
- Documentation
- Validation
- Management

... and will be the cornerstone for the quality of your project/product

Requirements Engineering - Get it right from the beginning

Techniques to get Requirements

Elicitation techniques



- Elicitation techniques serve the purpose of identifying the conscious, unconscious, and subconscious stakeholder requirements. However, there is no universal method to elicit these requirements

The most important influencing factors when choosing the appropriate elicitation techniques are as follows:

- the distinction between conscious, unconscious, and subconscious requirements that are to be elicited
- the time and budget constraints, as well as the availability of the stake- holders
- the experience of the requirements engineer with a particular elicitation technique
- the chances and risks of the project

General Groups of requirement elicitation methods



Method	Pros	Cons
Analysis of documentation	Fast source of information Independent of third parties	Not always available/complete/up-to-date
Interviews	Lots of information Individual perspective Allows to verify correctness of understanding	Takes lots of time and effort Limited amount of participants
Workshops	More/different participants in the same room Results immediately verifiable	Certain participants dominate Requires moderation
Questionnaires	Large number of participants Good for clearly defined questions	Not usable for open questions Often bad response ratio

Risks and problems



Stakeholders:

- Expectation and term discrepancies
- Know what they want, but aren't capable of describing it
- Don't know what they want
- Know what they want, but aren't capable of describing it
- Already fixed to a certain solution (not openminded)

Requirements Engineering:

- Clarification with stakeholder
- Risk analysis
- Build a common understanding
- Recognise conflicts and resolve existing conflicts
- Animate creativity



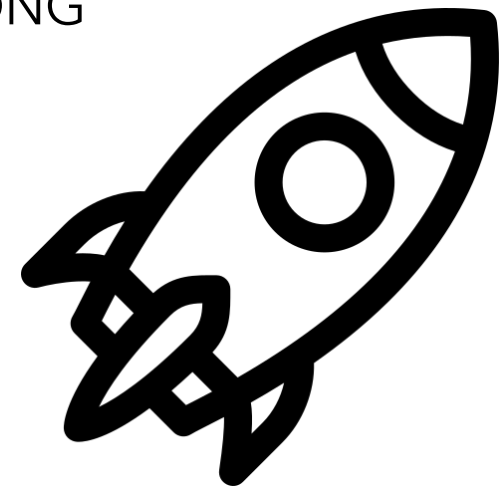
Requirement and innovation

«Provide the customer with exactly what they wish» → WRONG

«We know what fits best for the customer» → also WRONG

How can you force innovation?

- Propose innovative solutions to stakeholders
- Animate creativity on stakeholder
 - Future scenarios (create and play through)
 - Let all boundaries go / no limitations for once ;-)
 - Try to work with metaphors (search and explore)



→ It's not that easy to collect/create requirements. There's some work to do!

RE under pressure → Real World Scenario!



- Risk based / risk oriented specification
 - risk defines the effort not the time available
- Deepness / coverage not equally
 - Risky things more in detail, less risky less detailed
- Try (to go) for an incremental development/implementation
- Don't try to specify "perfectly", good enough will do



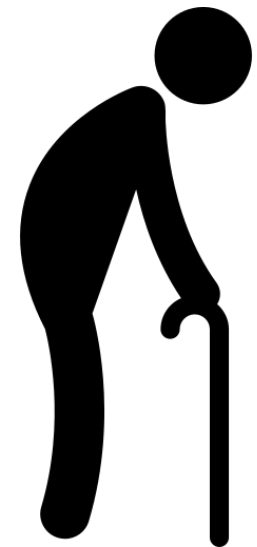


Eliciting Requirements

Old-School vs. more modern techniques



- Also in RE there are
 - More Static “old school” techniques to elicit requirements
- No right or wrong
 - Depending on your situation and project circumstances
 - Time
 - Stakeholders
 - Culture



Interview



- During an interview, the requirements **engineer asks** predetermined **questions** to one or more **stakeholders** and **documents the answers**. Questions that arise during the conversation can be discussed immediately, and the requirements engineer may uncover subconscious requirements through clever questions.
- Interviews can be employed during the entire development phase of the system. An experienced interviewer individually controls the course of the conversation, completely commits herself to each stakeholder, inquires about specific aspects, and thus ensures the completeness of the answers. The most prominent disadvantage of this elicitation technique is that it is very time-consuming.

[Requirements Engineering Fundamentals], p25 ff

Questionnaire



- Making use of **open and / or closed questions** (e.g., multiple choice questions) is another way of eliciting requirements from stakeholders. If there are a **large number of participants** that must be **surveyed**, an **online questionnaire** is a viable option. Questionnaires can **elicit a magnitude of information** in a **short amount of time** and at **low costs**.
- As long as answers are predetermined, even stakeholders that are not able to explicitly express their knowledge can deliver an assessment. A **disadvantage** of using a questionnaire is that it can be only employed to gather requirements the requirements engineer already knows or conjectures. **Creating a proper questionnaire** is often **tricky** and time-consuming and requires thorough knowledge of the domain in question and the psychological guidelines for creating questionnaires. In addition, as opposed to interviews, questionnaires do not provide immediate feedback between the surveyor and the surveyed, so it becomes apparent that questions were forgotten or badly formulated only once the questionnaires have been evaluated.

[Requirements Engineering Fundamentals], p25 ff

Creativity Techniques



- Creativity techniques serve the purpose of developing **innovative requirements**, delineating an initial vision of the system, and eliciting excitement factors. Creativity techniques are usually not well suited for establishing fine-grained requirements about the system behavior.

[Requirements Engineering Fundamentals], p25 ff

Brainstorming



- During brainstorming, **ideas are collected** within a **certain time frame**, usually in **groups of 5 to 10 people**. The ideas are **documented by a moderator** without discussing, judging, or commenting on them at first. Participants use ideas of other participants to develop new original ideas or to modify existing ideas. After that, the collected ideas are subjected to a thorough analysis. This technique is especially effective when a **large number of people of different stakeholder groups** are involved. Among the advantages of this technique is that a **large number of ideas can be collected in a short amount of time and multiple people** can expand on these ideas collaboratively. The unbiased collection of these ideas allows new solutions to pop up. Brainstorming is usually less effective when the dynamics of the group are muddled or when participants with very varied levels of dominance are involved.

[Requirements Engineering Fundamentals], p25 ff

What is 6-3-5?



- Kind of brainstorming technique (developed by Bernd Rohrbach)
- Also called Brain writing
- Difference to Brainstorming: people are not talking and it's not that chaotic



6 people

Teilnehmer	Idee 1	Idee 2	Idee 3
Teilnehmer 1			
Teilnehmer 2			
Teilnehmer 3			
Teilnehmer 4			
Teilnehmer 5			
Teilnehmer 6			



3 solutions



5 minutes

Advantages and Disadvantages



Advantages

- Method is easy to learn
- Direct feedback
- A lot of ideas in a short time
- Active participation from all members but avoids domination over introverts
- All ideas are recorded
- No moderator required
- No premature discussions

Disadvantages

- Stress due to time constraints
- Risk of similar ideas
- Reading handwriting or trouble summarising the own ideas
- All participants need to share a deep background knowledge

Lot more techniques



- Change of perspective
- Document Analysis
- Focus Groups
- Interface Analysis
- Observation
- Prototyping
- Requirements Workshops
- Survey / Questionnaire



Open / Closed Questions

Open Questions



- Open-ended questions are broad and can be answered in detail
 - What do you think about this product?
- W-Questions



Closed Questions



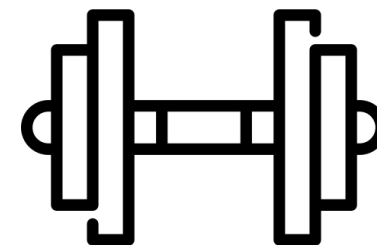
- closed-ended questions are narrow in focus and usually answered with a single word or a pick from limited multiple-choice options
 - Are you satisfied with this product?" → Yes/No/Mostly/Not quite





Ex: 4 People

- Phrase 5 open questions and 5 closed questions



114



Documentation of Requirements

Requirement Specification → Write it down!



- A requirements specification is a systematically represented collection of requirements, typically for a system or component, that satisfies given criteria.

Why do we specify?

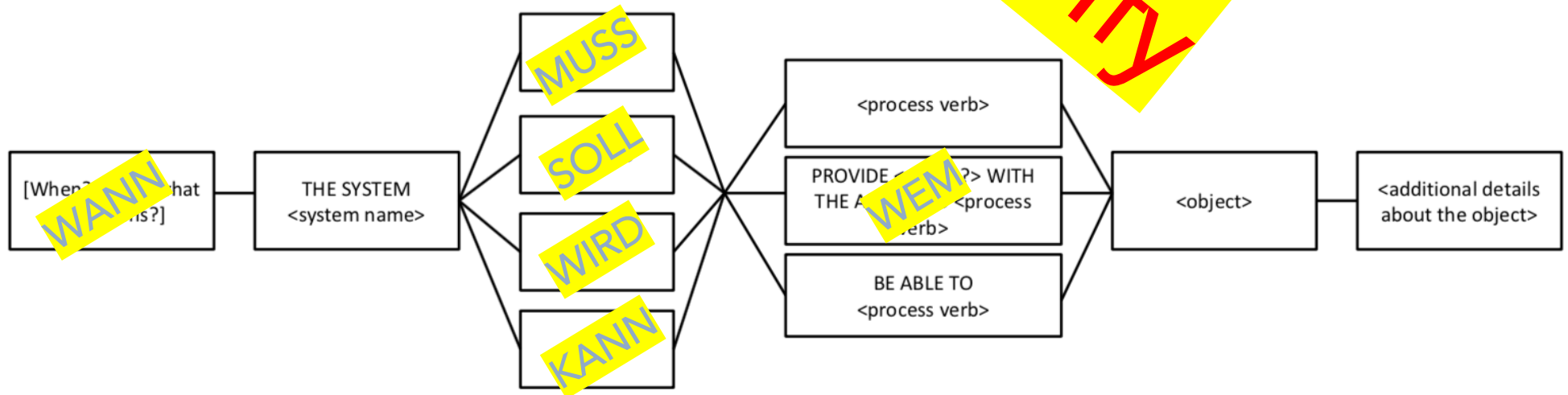
- Requirements are the **basis of the system development**.
- Requirements have a **legal relevance**.
- Requirements documents are complex
- Requirements must be accessible to all involved parties.



- [illegible]

If you used it → Be as clear as possible!

- complete requirements template with conditions



[Requirements Engineering Fundamentals], p57 ff

Activity of a System

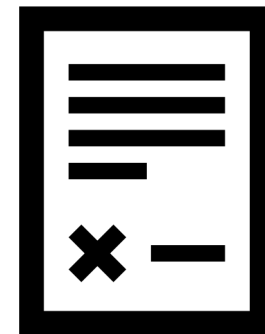


- **Autonomous** system activity
 - THE SYSTEM SHALL/SHOULD/WILL/MAY <process verb>
- **User** interaction
 - THE SYSTEM SHALL/SHOULD/WILL/MAY provide <whom?> with the ability to <process verb>
- **Interface** requirement
 - THE SYSTEM SHALL/SHOULD/WILL/MAY be able to <process verb>

Documentation using Conceptual Models



- In addition, conceptual models offer a decreased degree of ambiguity (i.e., fewer ways to be interpreted) than natural language due to their higher degree of formality.
- However, using conceptual modeling languages for requirements documentation requires specific knowledge of modeling.



Use case diagram

- A use case diagram allows you to gain a quick overview of the functionalities of the specified system. A use case describes which functions are offered to the user by the system and how these functions relate to other external interacting entities. However, use cases do not describe the responsibilities that the functions have in detail

Section	Content
Designation	UC-12-37
Name	Navigate to destination
Authors	John Smith, Sandra Miller
Priority	Importance for system success: high Technological risk: high
Criticality	High
Source	C. Warner (domain expert for navigation systems)
Responsible	J. Smith
Description	The driver of the vehicle types the name of the destination. The navigation system guides the driver to the desired destination.
Trigger event	The driver wishes to navigate to his destination.
Actors	Driver, traffic information server, GPS satellite system
Pre-condition	The navigation system is activated.
Post-condition	The driver has reached his destination.
Result	Route guidance to the destination
Main scenario	1. The navigation system asks for the desired destination. 2. The driver enters the desired destination. 3. The navigation system pinpoints the destination in its maps. 4. On the basis of the current position and the desired destination, the navigation system calculates a suitable route. 5. The navigation system compiles a list of waypoints. 6. The navigation system shows a map of the current position and shows the route to the next waypoint. 7. When the last waypoint is reached, the navigation system shows "destination reached" on the screen.
Alternative scenario	4a. Calculation of the route must honor traffic information and avoid traffic congestions. 4a1. The navigation system queries the server for updated traffic information. 4a2. The navigation system calculates a route that does not contain any traffic congestions.
Exception scenarios	Trigger event: The navigation system does not receive a GPS signal from the GPS satellite system.
Qualities	→ QR.04 (reaction time upon user input) → QR.15 (operating comfort) (QR = quality requirements)

[Requirements Engineering Fundamentals], p74 ff

Yet another template...

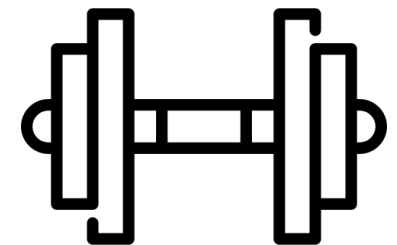


Use Case Name	ein Vorgang im System
Use Cases	verbundene Anwendungsfälle (Bezeichnung mit Subjekt + Verb; konkludieren)
System	enthält gesamte Software Systemname in Kasten darstellen
Akteur	▪ Systeme/Personen, welche mit System interagieren ▪ Löst Use Case aus
Trigger	▪ erste Aktion ▪ startet durch Nutzer, Zeit oder Fehler
Success Guarantees	Ziel der Aktion (laut Titel)

Clarify

Ex: 4 People

- Brainstorm --> Coffee vending machine
- Write 3 UC of a coffee vending machine



Try it...



Use Case Name	
Use Cases	
System	
Actor	
Trigger	
Success Guarantees	



Try it...



Use Case Name	
Use Cases	
System	
Actor	
Trigger	
Success Guarantees	

Try it...



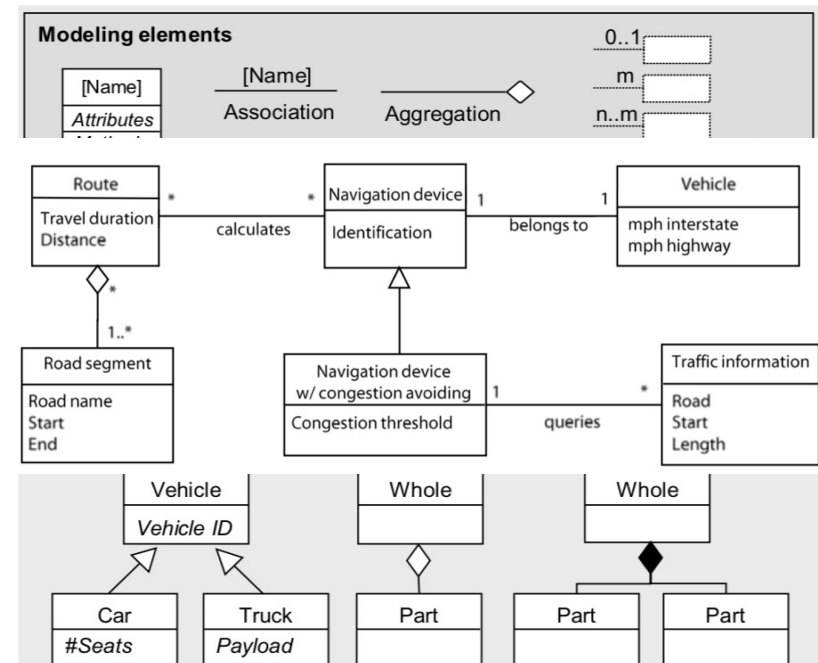
Use Case Name	
Use Cases	
System	
Actor	
Trigger	
Success Guarantees	





Class diagram

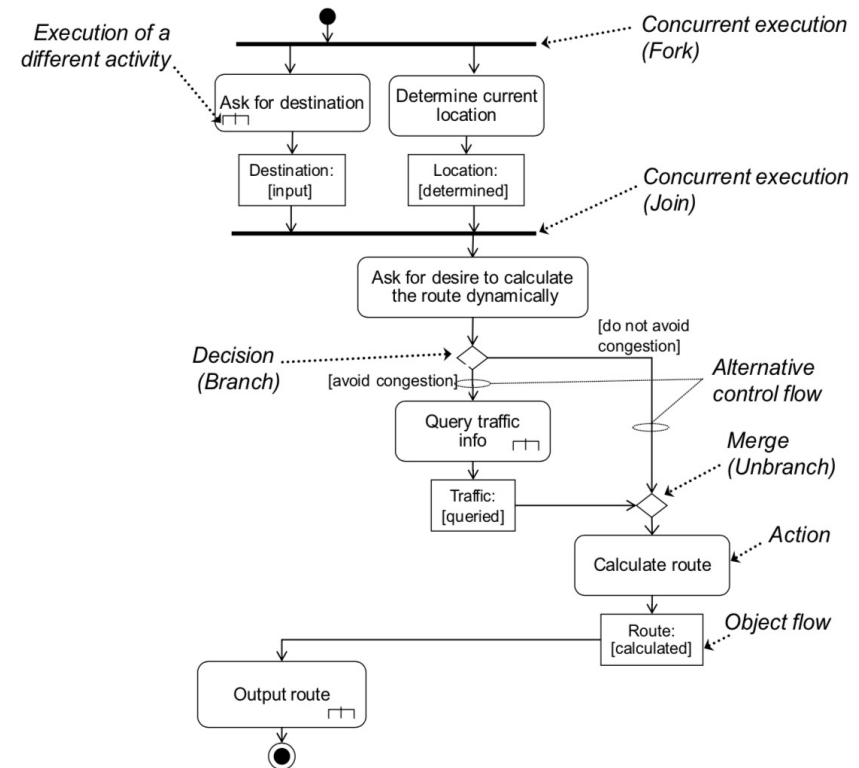
- Among other things, class diagrams are used in requirements engineering to document requirements with regard to the static structure of data, to document static-structural dependencies between the system and the system context, or to document complex domain terms in a structured manner
 - Class diagram in UML notation
 - Important modeling elements of class diagrams of UML



Activity diagram

- Using activity diagrams, business processes, or sequence-oriented dependencies of the system in regard to processes within the system context can be documented. Activity diagrams are also well suited to model the sequential character of use cases or to model a detailed specification of the interaction of functions that process data

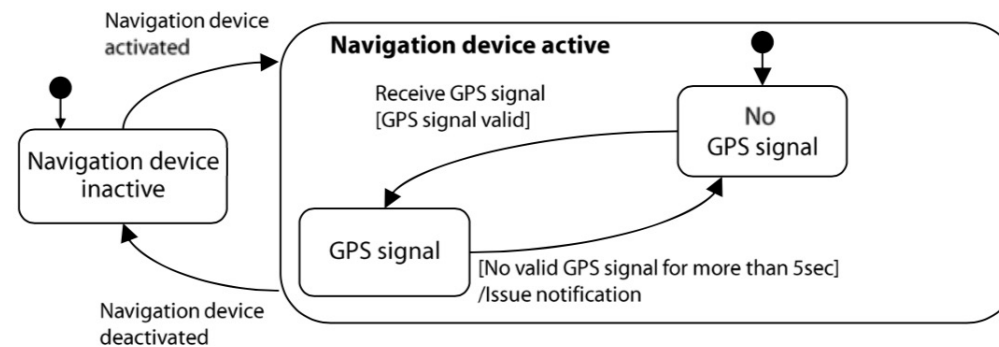
UML Activity Diagram



State diagram

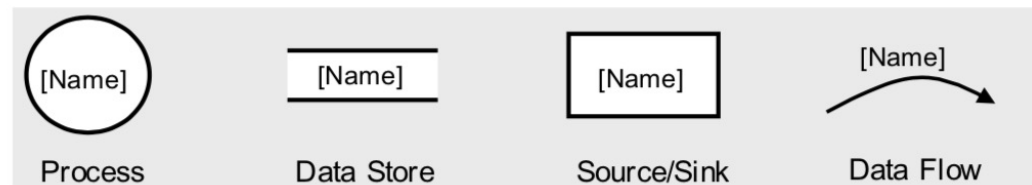


- State diagrams are used in requirements engineering to document event-driven behavior of a system. The focus of this type of model is on the individual states the system can be in, events and their respective conditions that trigger a state transition, and effects of the system in its environment.

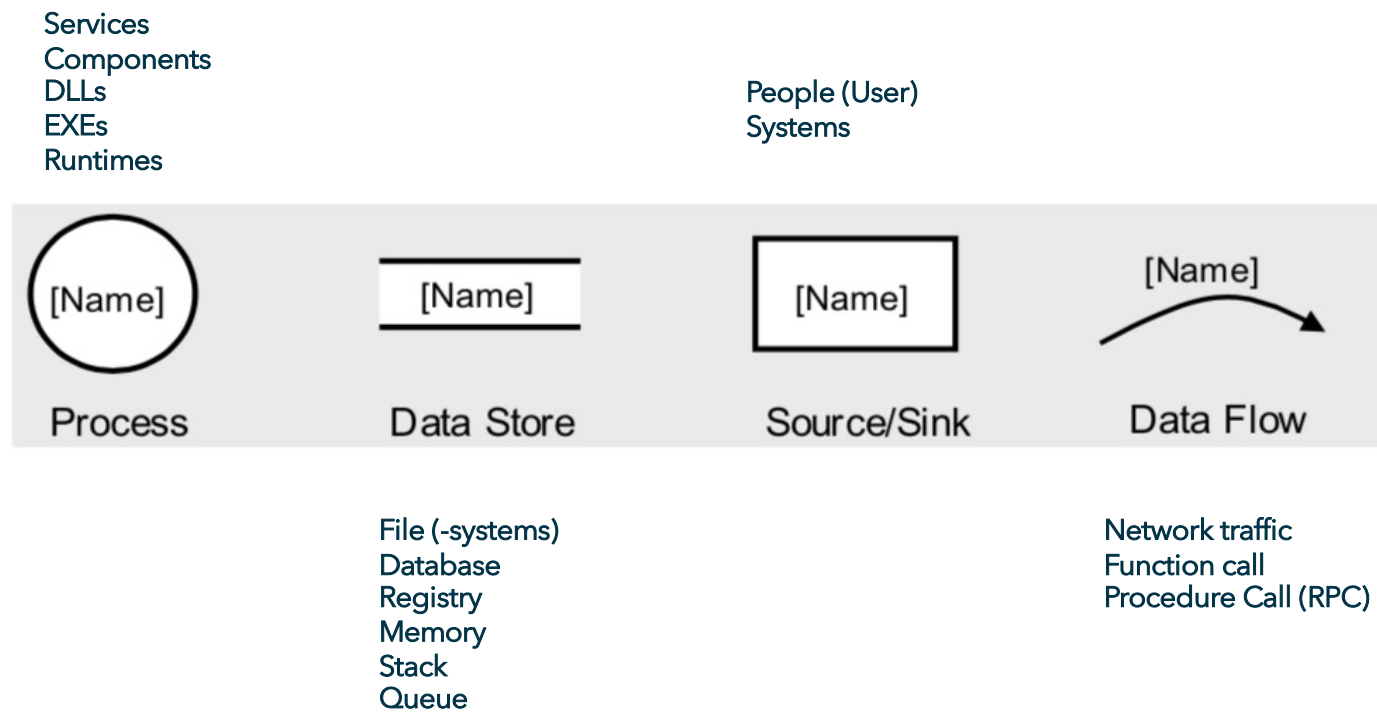


Dataflow diagram - Functional perspective

- The functional perspective of requirements deals with the transformation of input data received from the environment into output data released into the environment of the system.
- by means of
 - processes (functions)
 - data stores
 - Sources
 - sinks
- in the system environment as well

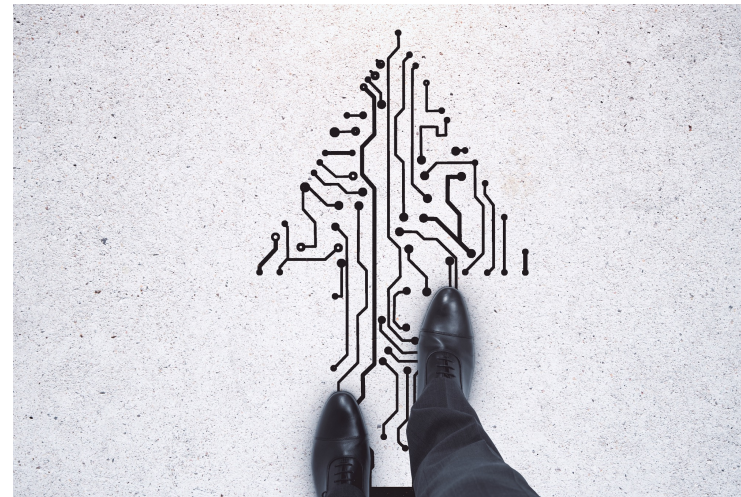


Notation of DFD



Create Diagrams

- Start with an overview which has:
 - A few external interactors
 - One or two processes
 - One or two data stores
 - Data flows to connect them
- Check your work
 - Can you tell a story without edits?
 - Does it match reality?



Diagrams: Trust Boundaries



- Add trust boundaries that intersect data flows
- Point/surface where an attacker can interject
 - Machine boundaries, privilege boundaries, integrity boundaries are examples of trust boundaries
 - Threads in a native process are often inside a trust boundary, because they share the same
 - privileges, rights, identifiers and access
- Processes talking across a network always have a trust boundary
 - They may create a secure channel, but they're still distinct entities
 - Encrypting network traffic is an 'instinctive' mitigation, but doesn't address tampering or spoofing

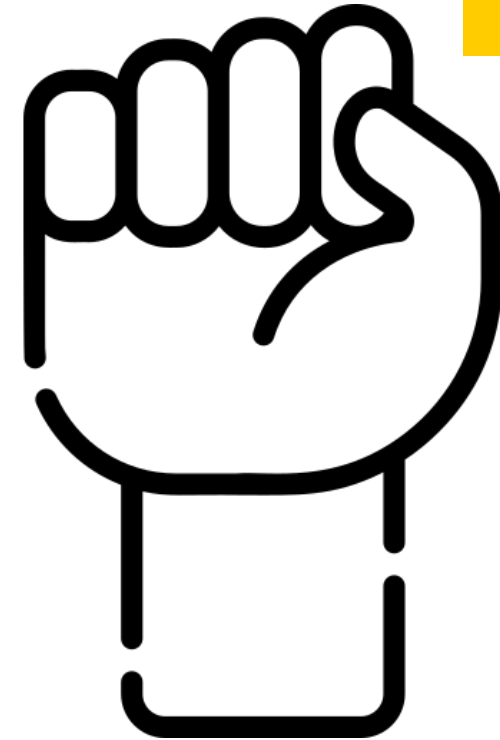
Diagram Iteration

- Iterate over processes, data stores, and see where they need to be broken down
- How to know it “needs to be broken down”
 - More detail is needed to **explain security impact** of the design
 - Object crosses a **trust boundary**
 - Words like “sometimes” and “also” indicate you have a combination of things that can be broken out
 - “Sometimes this data store is used for X”... probably add a second data store to the diagram



Rules of thumb - Cheat Sheet

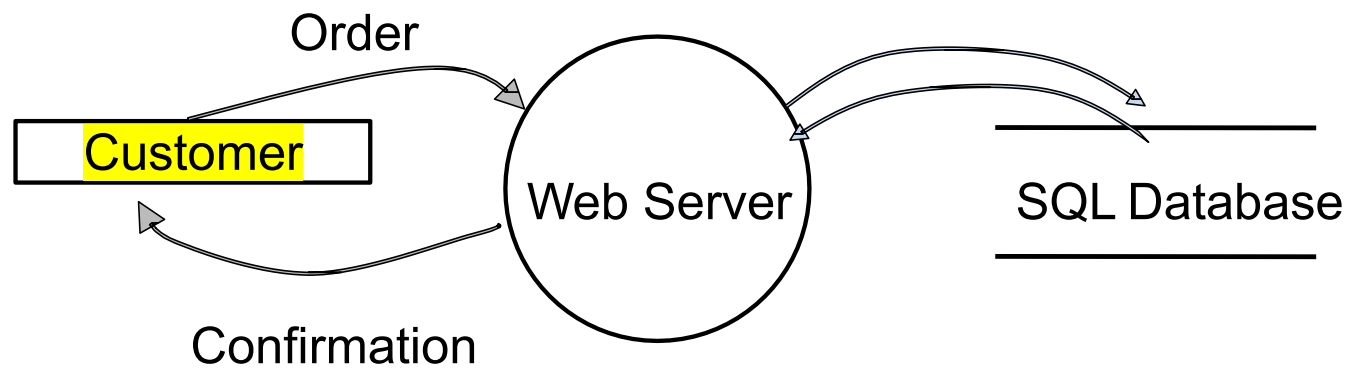
- Does data magically appear?
- Are there data sinks?
- Check: Data doesn't flow magically
- Check: "It" goes through a process



Does data magically appear?



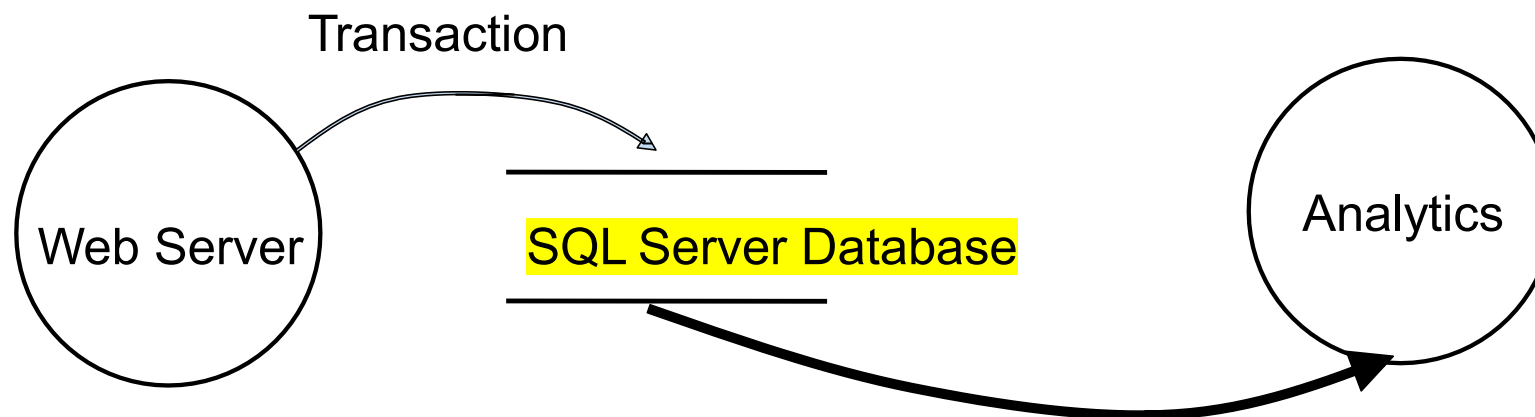
• .



Are there data sinks?



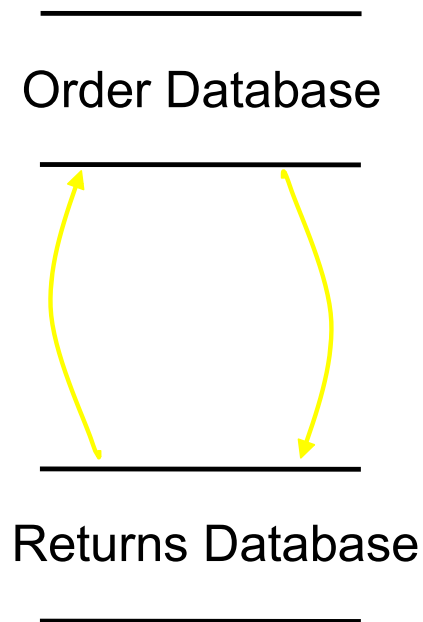
- .



Data doesn't flow magically



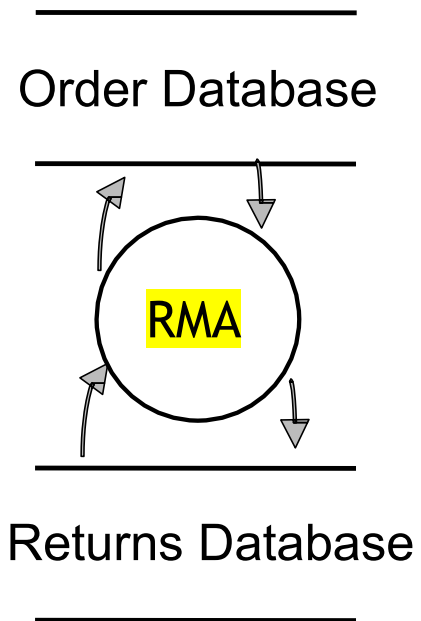
- .



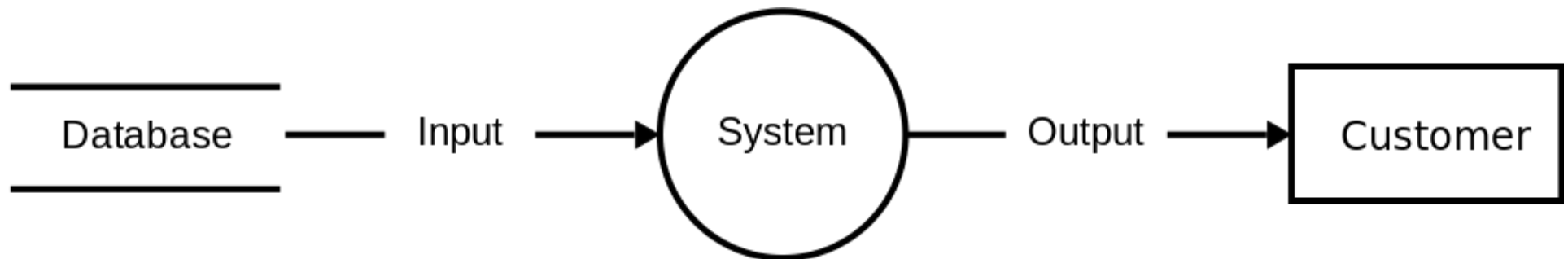
It goes through a process



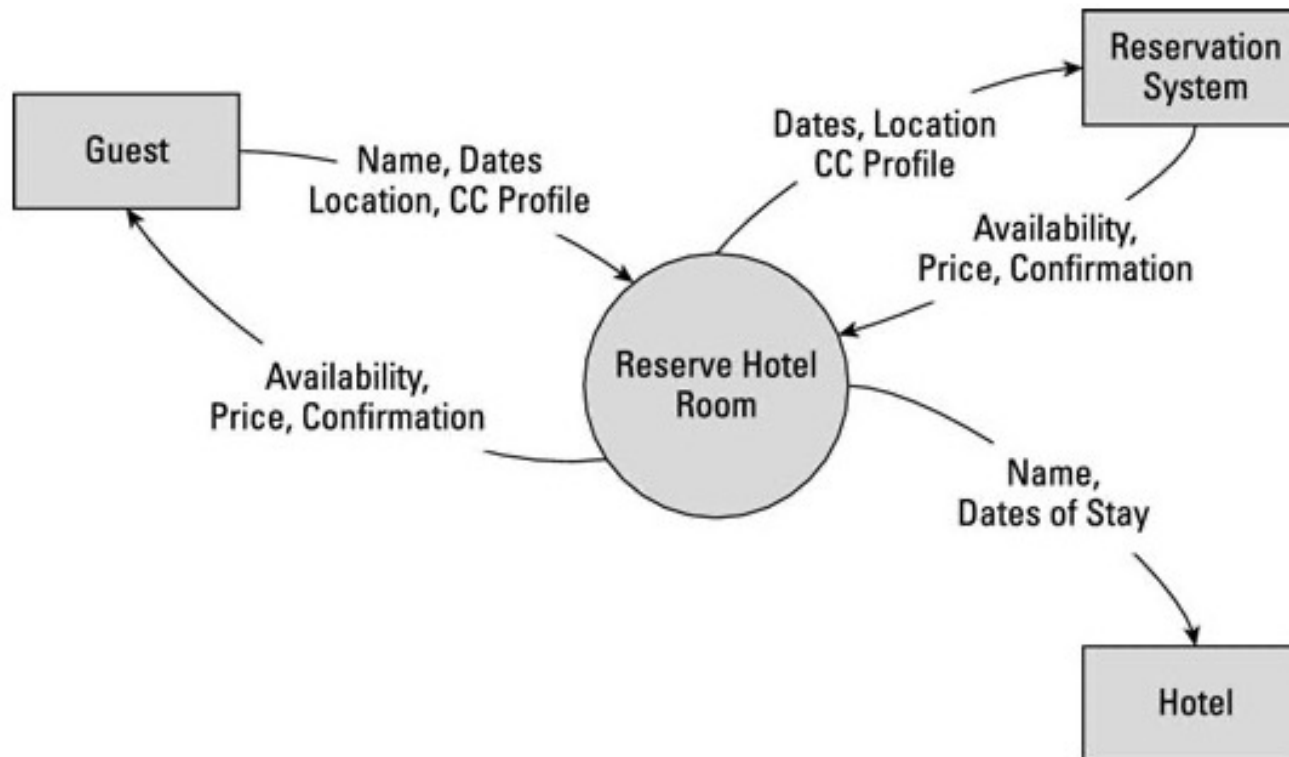
- .



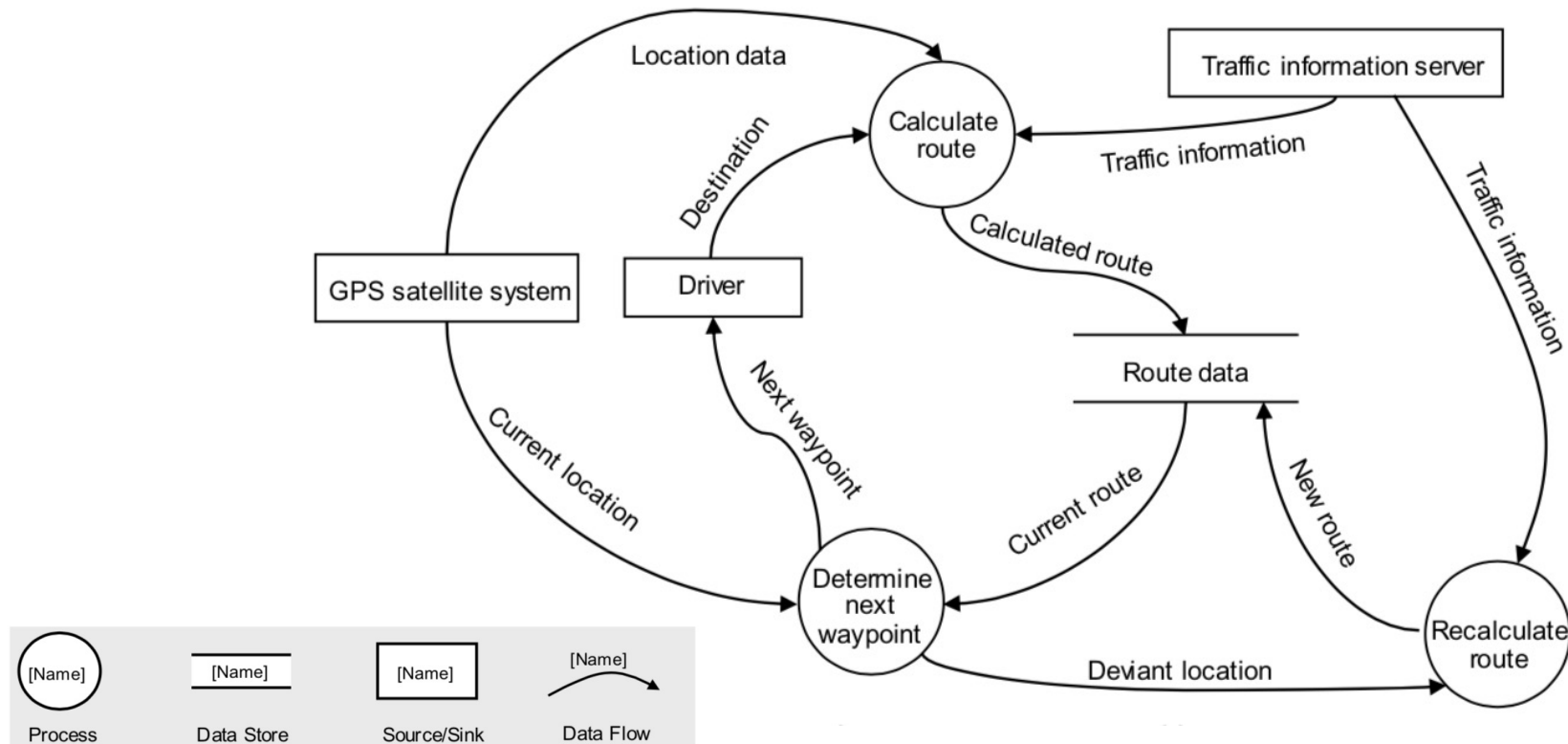
As simple as that...



As simple as that...



Dataflow diagram - Important for US!



Document Structures

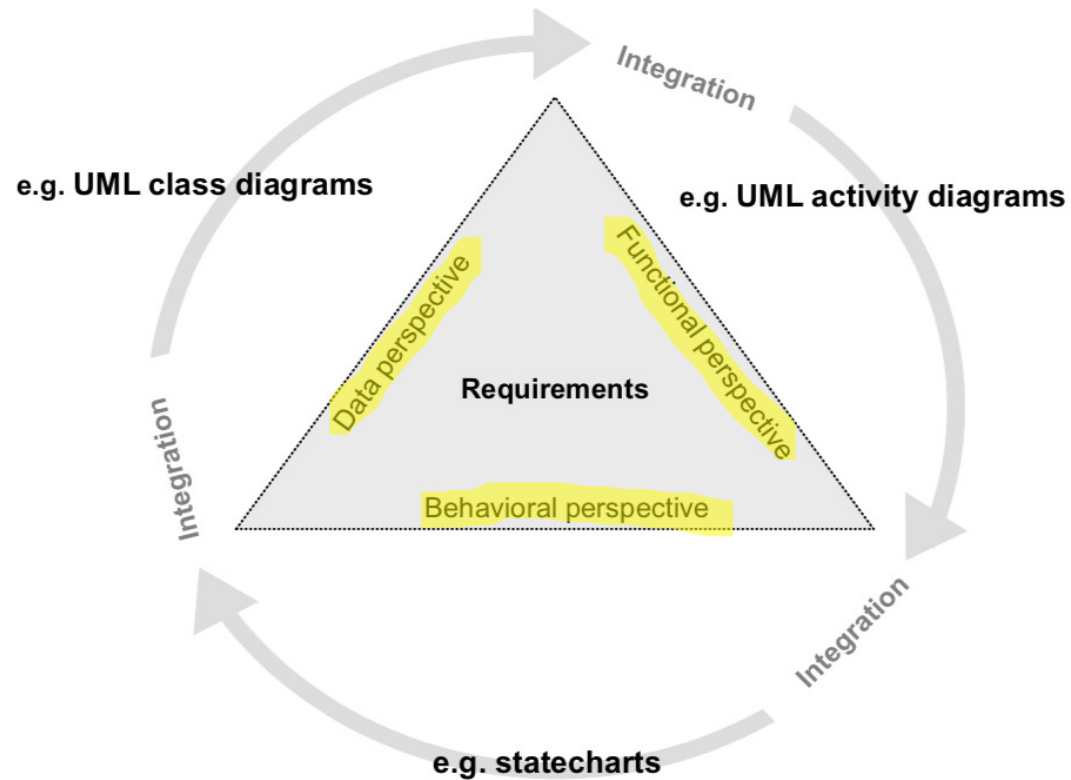


- **Standard** outlines offer a **predefined structure**, i.e., predefined stereotypes according to which the information can be classified

Advantages

- **simplify** incorporating new staff members.
- **quickly finding** desired contents.
- allow for **selective reading** and validation of requirements documents.
- simplified **reuse** of the contents of requirements documents.

Three types of perspectives





Distinguishes 3 types of perspectives

- **Data perspective:** In this perspective, the **structures of input and output data** as well as static-structural aspects of the usage and dependency relationships of the system in the system context are documented.
- **Functional perspective:** This perspective documents which information of the **system context is being manipulated by the system** to be developed and which data is being transmitted to the system context by the system.
- **Behavioral perspective:** The embedding of the system in the system context is documented **based on states** in this perspective. This is done, for instance, by documenting the reaction of the system to events within the system context, documenting the conditions that trigger a state change, or documenting the effects that the system has on its environment.

Quality criteria of requirement document



- Unambiguity and consistency
 - Clear structure
 - Modifiability and extendibility
 - Completeness
 - Traceability
-
- **Content:** Have all relevant requirements been elicited and documented with the appropriate level of detail?
 - **Documentation:** Are all requirements documented with respect to the predetermined guidelines for documentation and specification?
 - **Agreement:** Do all stakeholders concur with the documented requirements and have all known conflicts been resolved?

Don't forget the GLOSSARY



- A frequent **cause for conflicts** in requirements engineering is that the people that are involved in the development process have different interpretations of terms.
- Context-specific technical terms
- Abbreviations and acronyms
- Everyday concepts that have a special meaning in the given context
- Synonyms, i.e., different terms with the same meaning
- Homonyms, i.e., identical terms with different meanings

Use of standards



- **simplify** incorporating new members
- **outlines** allow for quickly finding desired contents.
- allow for **selective reading** and validation of requirements documents.
- **automatic verification** of requirements documents (e.g., regarding to completeness).
- **simplified reuse** of the contents of requirements documents.

Next: 3 of the **most widely used** standardized document structures

Rational Unified Process (RUP)

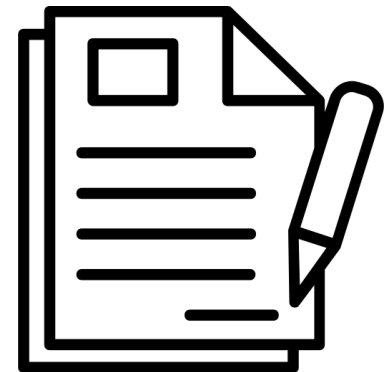


- is usually used for software systems that are developed using object-oriented methods. The client creates a **business model** that contains different artifacts from the business environment (e.g., business rules, business use cases, business goals), which serve as the basis for requirements of the system over the course of development. The contractor uses the structures of the **software requirements specification (SRS)** to document all software requirements. These structures are closely related to the ISO/IEC/IEEE standard 29148:2011.

SRS Examples



- <https://www.oasis-open.org/committees/download.php/14981/EDXLRqmtsSupplement101905.doc>
- https://gephi.org/users/gephi_srs_document.pdf
- <https://www.uccs.edu/Documents/tboults/srs.doc>



ISO/IEC/IEEE standard 29148:2011



Contains an outline designed for the documentation of software requirements (software requirements specification). The standard structure suggests dividing the requirements document into **five parts** with regard to their subject matter:

- A chapter with **introductory information** (e.g., system goal, system bounding) and a general description of the software (e.g., perspective of the system, properties of future users)
- A chapter with a **listing of all documents that are referenced** in the specification
- A chapter for **specific requirements** (e.g., functional requirements, performance, interfaces)
- A chapter with all **planned measures for verification**
- **Appendices** (e.g., information about assumptions that were made, identified dependencies)

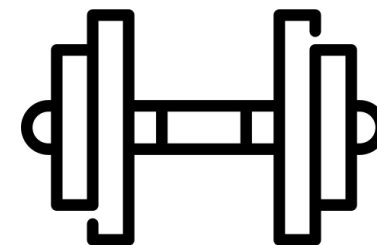
V-Model [V-Modell 2004]



- German Federal Ministry of the Interior (BMI) defines different structures, depending on the creator of the requirements document:
 - The **Customer Requirements Specification**, known in the German original as **Lastenheft**, is created by the customer and describes all of the demands to the contractor regarding the subject of the contract, i.e., deliveries and services. In addition, in many cases, demands of the users, including all constraints to the system and the development process, are documented. Therefore, the Customer Requirements Specification usually describes what is made for what.
 - The **System Requirements Specification**, known in the German original as **Pflichtenheft**, is based on the Customer Requirements Specification and contains the implementation suggestions that the contractor has elaborated. Therefore, the System Requirements Specification is a refinement of the requirements and constraints of the Customer Requirements Specification.



Ex: Download an SRS Doc and have a look at it
→ Security section?



160



Specification review, validate and approval SRS

Review rounds



- Internal SRS review
 - Before “releasing” an SRS to the customer
 - Candidates: TPM, BPM, other BA, tester
 - Doc Team (for proof reading)
- Document Versioning example
 - <1 - work in progress
 - 1.0 - released (initial version)
 - >1 - released (further versions)



Validation



Considering the following six principles of requirements validation **increases the quality** of the validation results:

- Principle 1: Involvement of the correct stakeholders
- Principle 2: Separating the identification and the correction of errors
- Principle 3: Validation from different views
- Principle 4: Adequate change of documentation type
- Principle 5: Construction of development artifacts
- Principle 6: Repeated validation

SRS Approval



- Get the **list of acceptors** from the PM/customer
- If possible, organize an SRS walkthrough meeting with the acceptors
- The customer must fill out a **sign-off protocol**
 - final remarks/feedback, conditions for sign-off
- Before incorporating the requested changes, send a list of the **review** items to the acceptors (incl. those that did not attend the meeting) - (last chance to object)
- Incorporate the changes (save as a new version) and **send it to the customer**



RE must support Implementation & Testing

Implementation



- Before development
 - Consider organizing an SRS walkthrough with the devs
- During development
 - Your main task is to answer developers' questions
 - Large number of questions is an indication for insufficient/unclear specification
 - Certain open points will need additional input from customer
 - Watch out for potential Change Requests
 - Every time you clarify or answer questions, make sure that the information gets documented in the SRS
 - Don't forget to increment the version number (for traceability)

Testing



- Internal
 - Support testers in creating test cases
 - Answer questions (and update SRS if necessary)
 - Testers might identify unspecified (but valid) scenarios
 - Clarify the expected behavior with customer
 - Update the SRS
- External
 - Make sure that the reported defects are valid
 - Some "defects" will in fact be Change Requests
 - CRs have to be tracked separately, as they usually have to be separately planned (and billed). Clarify the handling of CRs with the BPM.



Maintain

Keep them up2date

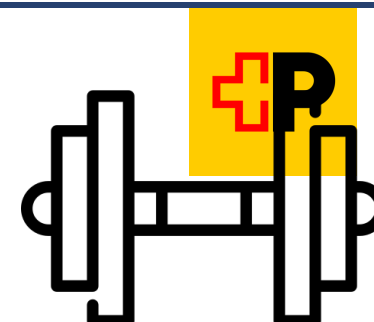
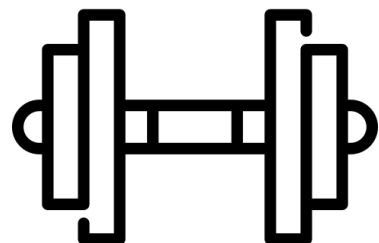


- React to actual situation
- Reflect changes on environment
 - Stakeholder
 - System boundaries
 - Other circumstances

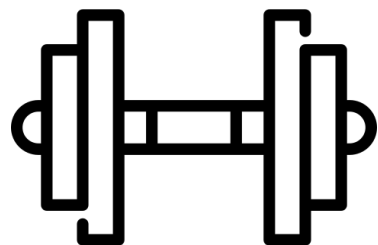
Importance



- Budget
- Delivery (time) / Schedule
- Quality (testing)
- Scope / Feature problem

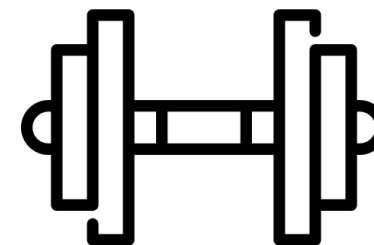


Exercise-Time...



Swiss Post Cybersecurity AG, Aarau

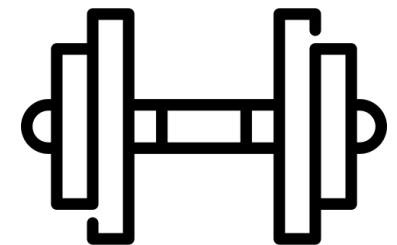
www.security.ch



172



Ex: Create a DFD of a coffee vending machine



Threat Modeling Tools



- Microsoft Threat Modeling Tool 2016
 - <https://www.microsoft.com/en-us/download/details.aspx?id=49168>
- OWASP Threat Dragon
 - https://www.owasp.org/index.php/OWASP_Threat_Dragon